Course Project

Elective – I (Data Science Foundation)

Smart Visualization Assistant

Submitted To – Prof. Deepak Yadav Sir
Submitted By – Gourav Singh Panwar (AU23B1009)

1. Introduction –

- Data visualization is a critical step in the data analysis process. Beginners often struggle to select appropriate charts, leading to incorrect interpretations. This project presents a **rule-based visualization assistant** that automatically analyzes a dataset and recommends a suitable visualization with a short explanation.

- The focus is on **simplicity, interpretability, and automation** rather than complex machine learning.

2. Problem Statement –

- Beginners in data science often struggle to choose the correct visualization for their dataset. Incorrect graph selection can misrepresent patterns and relationships. There is a need for a system that can automatically analyze the dataset and suggest suitable visualizations.

3. Scope –

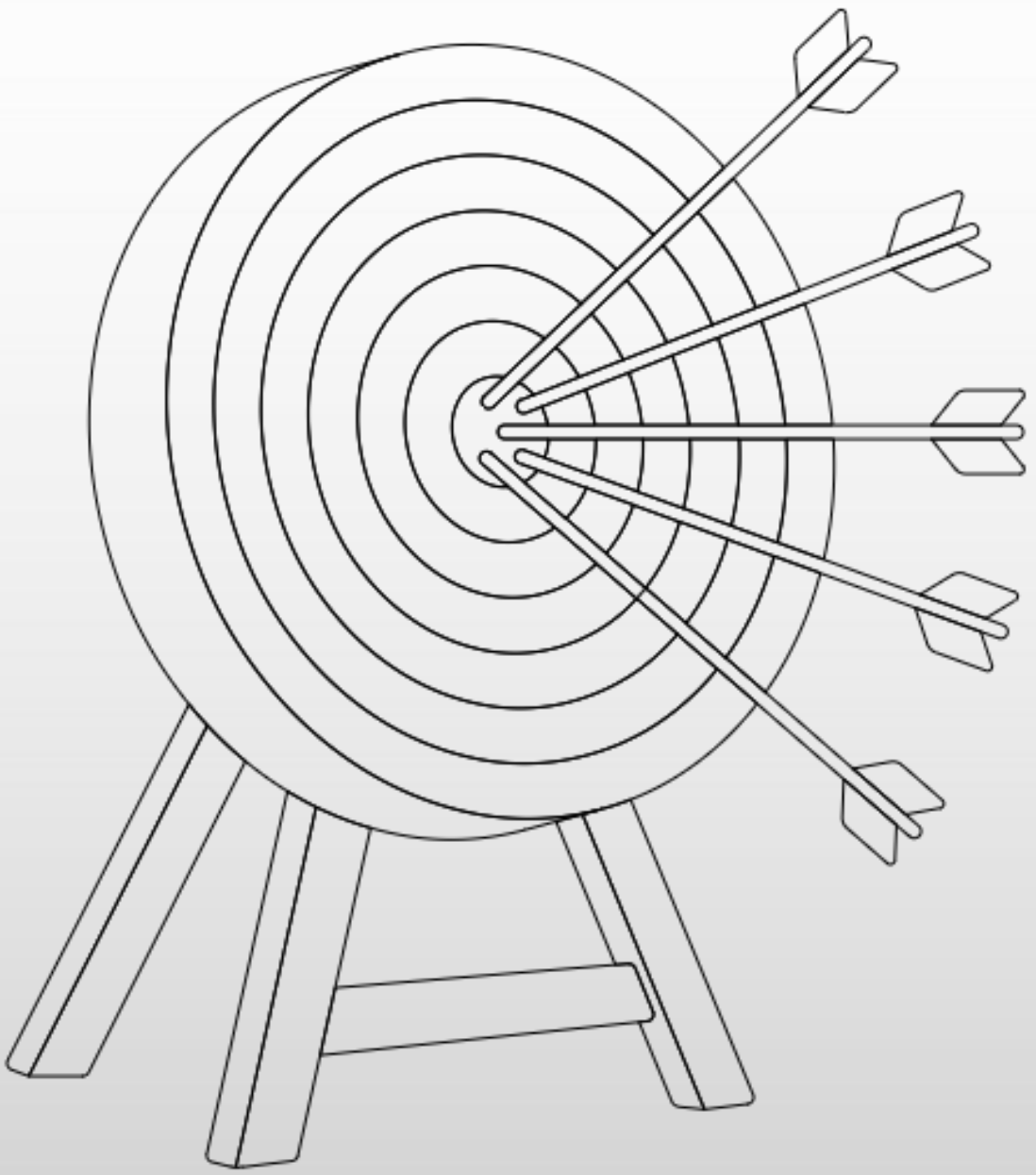
1. Applicable for –

i. Beginners in data science
ii. Small to medium datasets
iii. Exploratory data analysis

2. Out of scope –

i. Advanced multi-variable analytics
ii. Domain-specific reasoning
iii. Machine learning-based recommendations

4. Objectives –



Automatic Visualization

Generates visualizations automatically

Visualization Explanation

Explains the chosen visualization

Visualization Recommendation

Recommends appropriate visualizations

Data Type Detection

Detects data types

Dataset Structure Analysis

Analyzes dataset structure

5. Methodology –

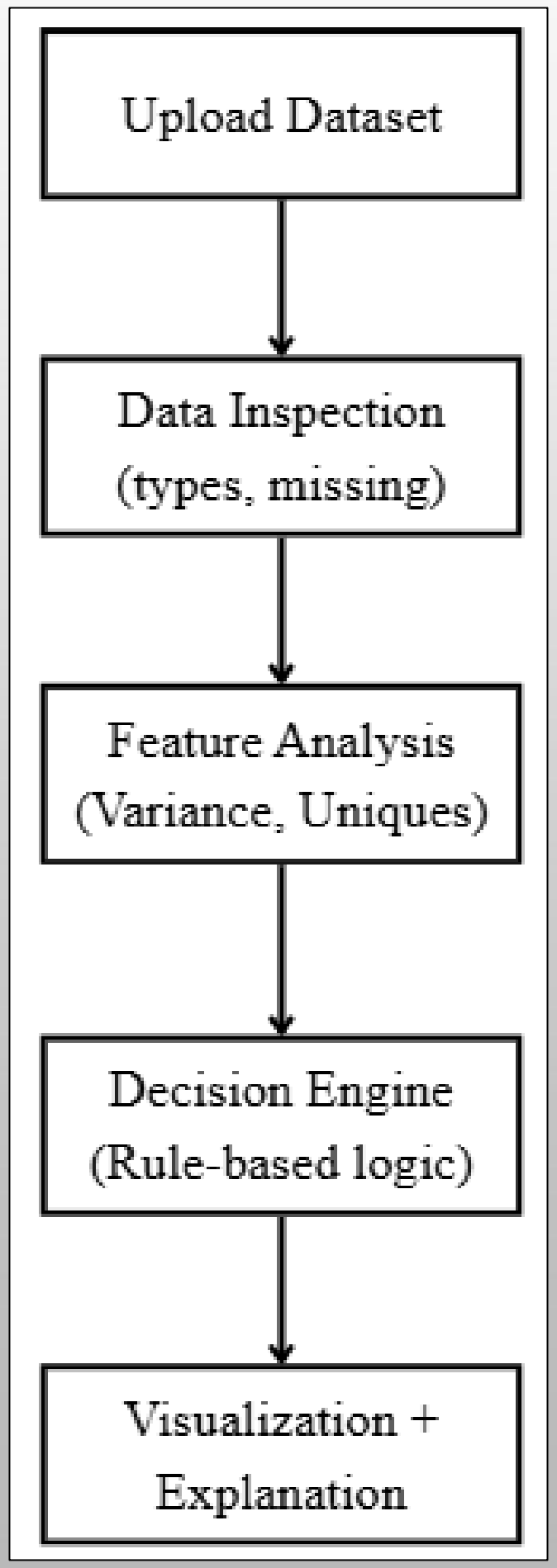
1. Upload dataset (CSV/Excel)
2. Load data using Pandas
3. Detect numeric and categorical columns
4. Compute basic statistics (variance, unique count)
5. Apply decision rules

6. Generate visualization using Matplotlib

7. Display explanation (insight)

6. System Architecture –

6.1) Flow Diagram –



7. Core Logic (Decision Engine) –

- The system uses simple and explainable rules –

Case 1 – Single Numeric Variable

i. Condition – Only one numeric column
ii. Selection – That column
iii. Visualization – Histogram
iv. Reason – Shows distribution

Case 2 – Single Categorical Variable

i. Condition – Only one categorical column
ii. Selection – That column
iii. Visualization – Bar Chart
iv. Reason – Shows frequency comparison

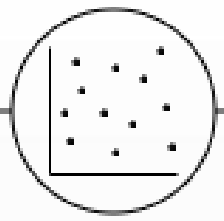
Case 3 – Multiple Numeric Variables

i. Condition – Two or more numeric columns
ii. Logic – Select top 2 columns with **highest variance**
iii. Visualization – Scatter Plot
iv. Reason – Shows relationship between most informative variables

Case 4 – Mixed Data (Numeric + Categorical)

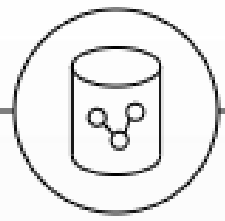
i. Numeric → highest variance
ii. Categorical → limited unique values (≤10)
i. Visualization – Bar Chart (mean aggregation)
ii. Reason – Compare numeric trends across categories

8. Key Concepts –



Variance

- Measures spread of data
- High variance → more informative



Unique Value Count

- Used for categorical columns
- Not directly applicable

9. Implementation Details –

1. Frontend (GUI) –

i. Built using Tkinter
ii. File upload button
iii. Dataset info display
iv. Analyze & Visualize button

2. Backend –

i. Pandas for data processing
ii. Matplotlib for visualization
iii. Rule-based decision logic

10. Example Walkthrough –

1. Input Dataset (Cricket Data) –

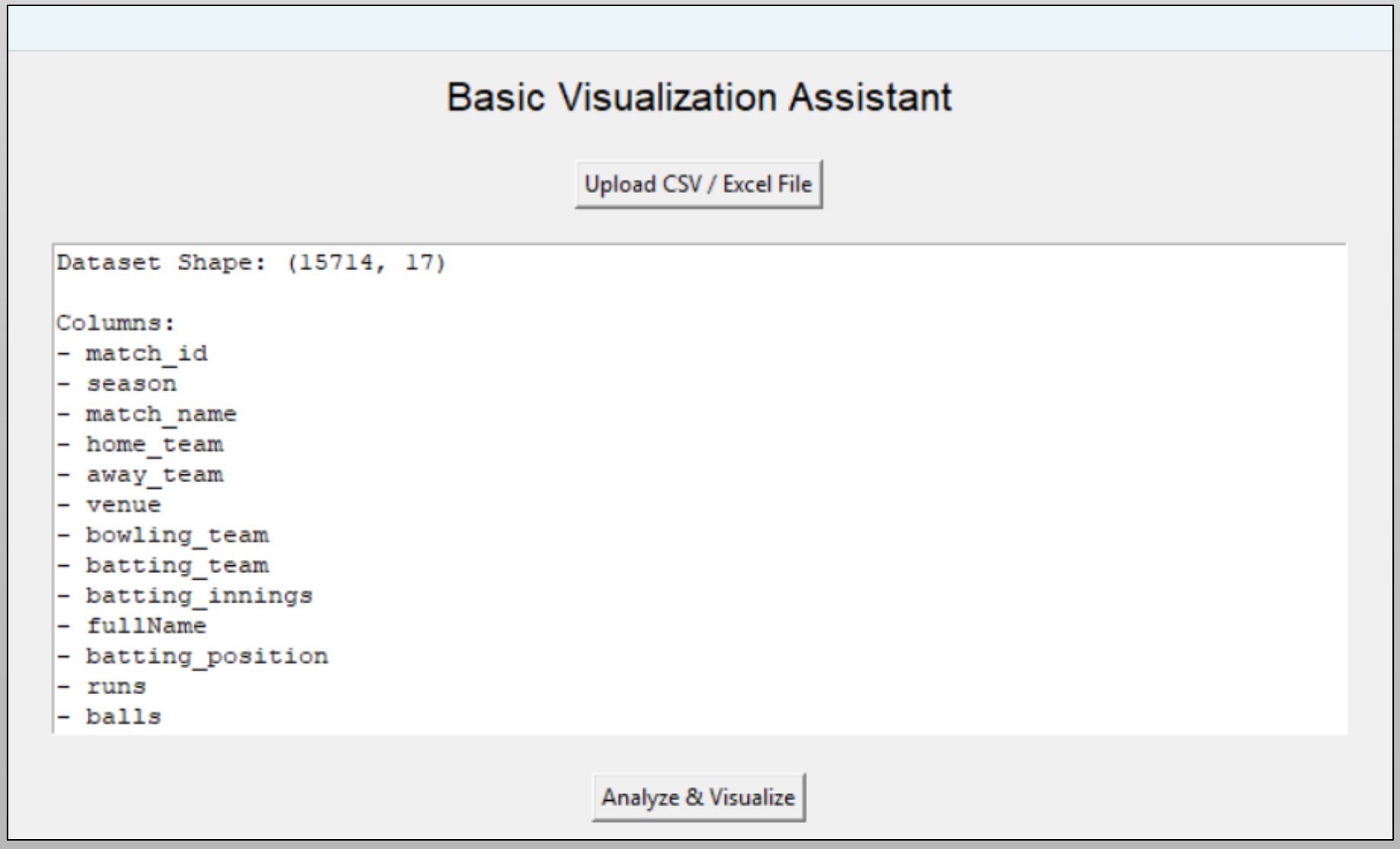
i. Shape – (15714, 17)
ii. Numeric Columns – runs, balls, strike_rate, fours, sixes
iii. Categorical Columns – player names, teams, venue

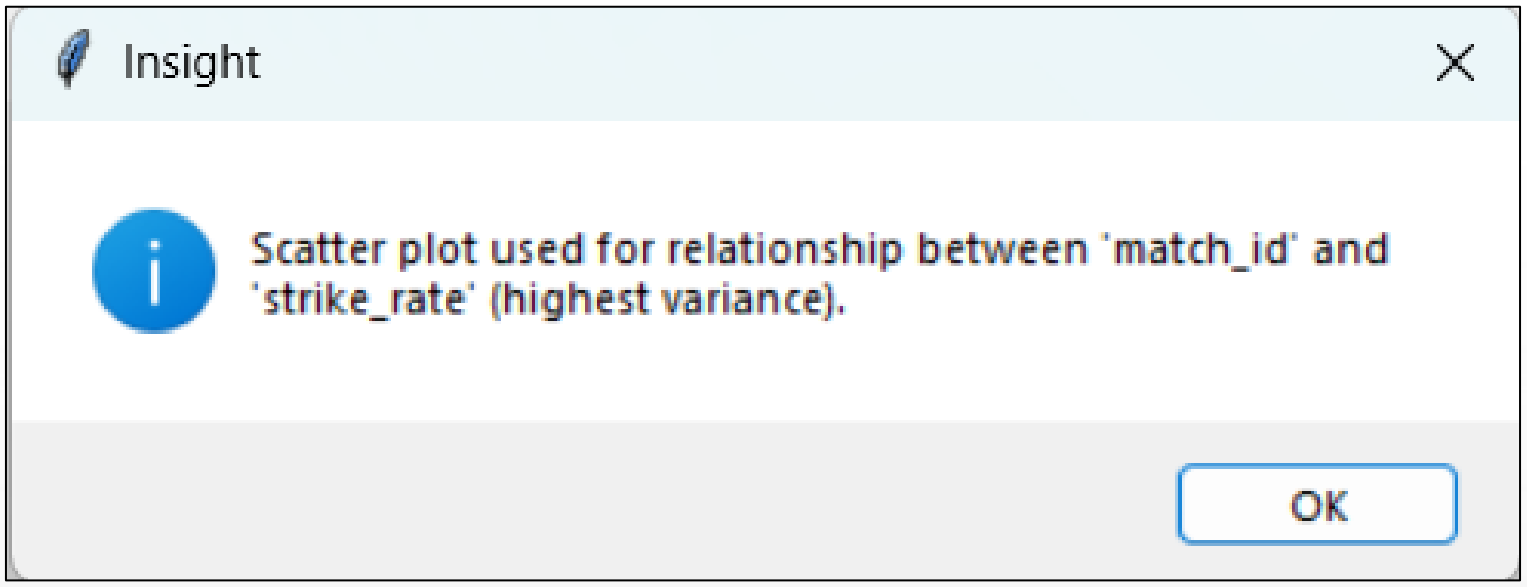
2. System Behavior

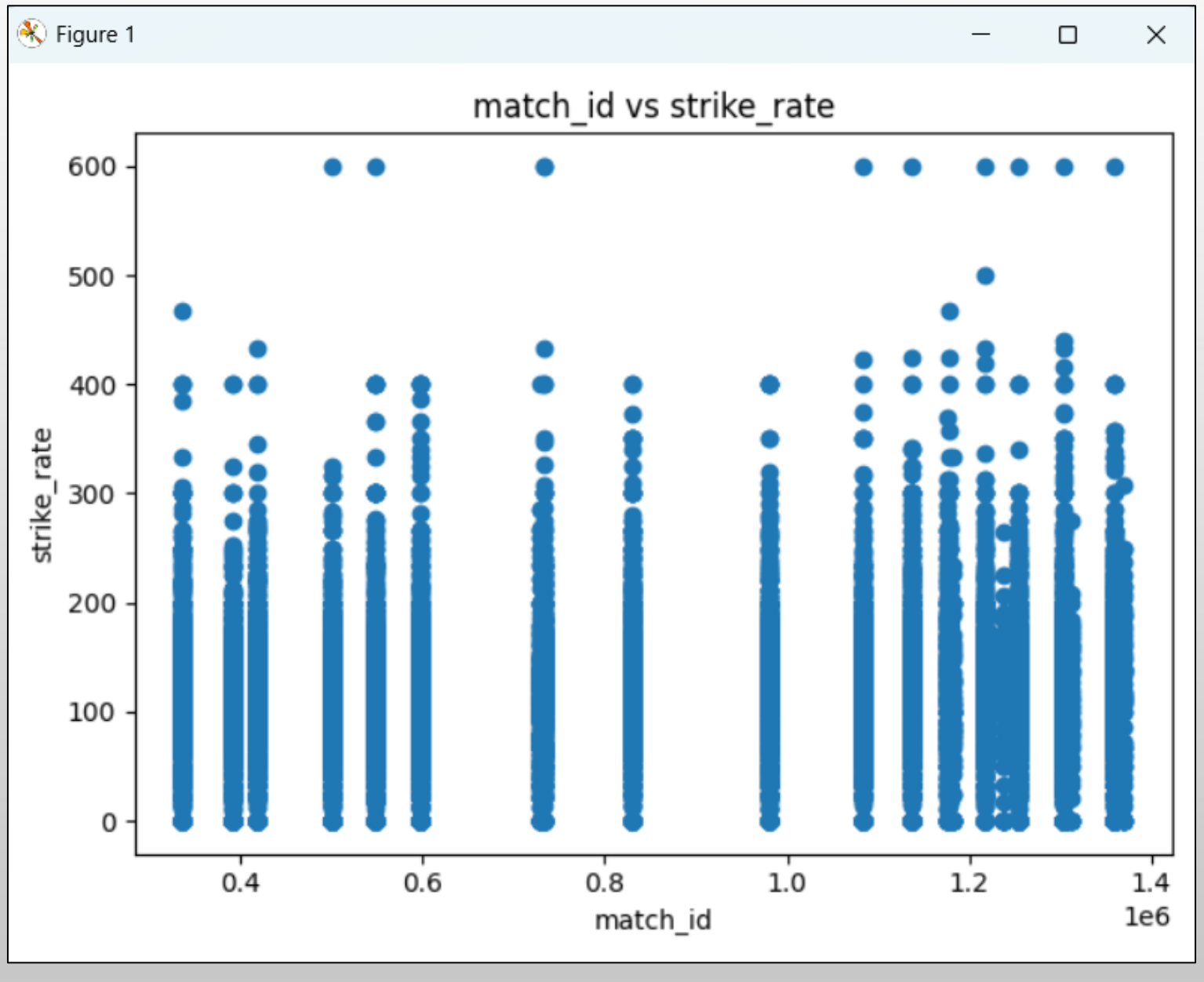
i. Detected multiple numeric columns
ii. Computed variance
iii. Selected top 2 columns (e.g., runs and balls)
iv. Generated scatter plot

3. Output Insight

- "Scatter plot used to show relationship between runs and balls, as they have high variance and are informative."







11. Results –

- The system successfully –

i. Loads datasets
ii. Detects structure
iii. Selects appropriate visualization
iv. Generates explanation

12. Advantages –

i. Simple and beginner-friendly
ii. Fully automatic
iii. No manual chart selection
iv. Easy to explain logic

13. Limitations –

i. No domain understanding
ii. Limited chart types
iii. Basic statistical logic only

14. Future Enhancements –

i. Add time-series detection (line chart)
ii. Add box plots and heatmaps
iii. Improve selection using ML
iv. Add interactive dashboard

15. Conclusion –

- This project demonstrates a practical and simple approach to automatic visualization using rule-based logic. It provides a strong foundation for beginners and can be extended into more advanced systems.

16. Code Explanation –

- This section explains the important parts of the implementation in a simple and clear way.

16.1 File Upload Function –

i. Uses file dialog to select CSV or Excel file
ii. Reads file using Pandas (read_csv / read_excel)
iii. Stores dataset in a global variable df

16.2 Dataset Information Display –

i. Shape of dataset
ii. Column names
iii. Data types
iv. Missing values

- Helps user understand structure before visualization

16.3 Data Type Detection –

```
numeric_cols = df.select_dtypes(include=['int64', 'float64']).columns
categorical_cols = df.select_dtypes(include=['object']).columns
```

i. Separates numeric and categorical columns
ii. Forms the base for decision logic

16.4 Variance Calculation (Numeric Selection) –

```
variances = df[numeric_cols].var().sort_values(ascending=False)
```

i. Computes variance of numeric columns
ii. Sorts columns by importance
iii. Selects top columns for visualization

16.5 Unique Value Check (Categorical Selection) –

if df[col].nunique() <= 10:

i. Ensures categorical column is not too large
ii. Avoids cluttered bar charts

16.6 Decision Logic Function –

i. Uses conditional statements (if-elif)
ii. Matches dataset structure to visualization type
iii. Implements 4 main cases –

a. Single numeric → Histogram
b. Single categorical → Bar chart
c. Multiple numeric → Scatter plot
d. Mixed → Grouped bar chart

16.7 Visualization Generation –

- Uses Matplotlib functions -

i. plt.hist() for histogram
ii. plt.scatter() for scatter plot
iii. plot(kind='bar') for bar chart

- Adds –

i. Title
ii. Axis labels
iii. Layout adjustment

16.8 Insight Messages –

i. Uses message boxes to explain why a graph is selected
ii. Helps beginners understand reasoning

➢ Code –

```
import tkinter as tk
from tkinter import filedialog, messagebox
import pandas as pd
import matplotlib.pyplot as plt


# Create main window
root = tk.Tk()
root.title("Basic Visualization Assistant")
root.geometry("750x520")

df = None

# ---------------- FILE UPLOAD ----------------
def upload_file():
    global df

    file_path = filedialog.askopenfilename(
        filetypes=[("CSV files", "*.csv"), ("Excel files", "*.xlsx")]
    )

    if not file_path:
        return

    try:
        if file_path.endswith(".csv"):
            df = pd.read_csv(file_path)
        else:
            df = pd.read_excel(file_path)

        messagebox.showinfo("Success", "File loaded successfully!")
        show_basic_info()

    except Exception as e:
        messagebox.showerror("Error", str(e))


# ---------------- DATASET INFO ----------------
def show_basic_info():
    global df
```

```
    if df is None:
        return


    info_text.delete("1.0", tk.END)


    info_text.insert(tk.END, f"Dataset Shape: {df.shape}\n\n")


    info_text.insert(tk.END, "Columns:\n")
    for col in df.columns:
        info_text.insert(tk.END, f"- {col}\n")


    info_text.insert(tk.END, "\nData Types:\n")
    info_text.insert(tk.END, f"{df.dtypes}\n")


    info_text.insert(tk.END, "\nMissing Values:\n")
    info_text.insert(tk.END, f"{df.isnull().sum()}\n")



# ---------- MAIN LOGIC ----------
def analyze_and_visualize():
    global df


    if df is None:
        messagebox.showwarning("Warning", "Upload a file first!")
        return


    numeric_cols = df.select_dtypes(include=['int64', 'float64']).columns
    categorical_cols = df.select_dtypes(include=['object']).columns


    plt.figure()


    # -------- Case 1: Only ONE numeric column --------
    if len(numeric_cols) == 1 and len(categorical_cols) == 0:
        col = numeric_cols[0]


        plt.hist(df[col], bins=20)
        plt.title(f"{col} Distribution")
        plt.xlabel(col)
        plt.ylabel("Frequency")
```

```
    messagebox.showinfo("Insight",
        f"Histogram used because dataset has one numeric variable ({col}).")


# -------- Case 2: Only ONE categorical column --------
elif len(categorical_cols) == 1 and len(numeric_cols) == 0:
    col = categorical_cols[0]


    df[col].value_counts().plot(kind='bar')
    plt.title(f"{col} Count")


    messagebox.showinfo("Insight",
        f"Bar chart used because dataset has one categorical variable ({col}).")


# -------- Case 3: Multiple numeric columns --------
elif len(numeric_cols) >= 2:
    # Select top 2 columns with highest variance
    variances = df[numeric_cols].var().sort_values(ascending=False)
    col1 = variances.index[0]
    col2 = variances.index[1]


    plt.scatter(df[col1], df[col2])
    plt.xlabel(col1)
    plt.ylabel(col2)
    plt.title(f"{col1} vs {col2}")


    messagebox.showinfo("Insight",
        f"Scatter plot used for relationship between '{col1}' and '{col2}' (highest variance).")


# -------- Case 4: Mixed (numeric + categorical) --------
elif len(numeric_cols) >= 1 and len(categorical_cols) >= 1:
    # Select numeric with highest variance
    num_var = df[numeric_cols].var().sort_values(ascending=False).index[0]


    # Select categorical with limited categories (avoid clutter)
    cat_var = None
    for col in categorical_cols:
        if df[col].nunique() <= 10:
            cat_var = col
            break


    if cat_var:
        grouped = df.groupby(cat_var)[num_var].mean()
```

```
            grouped.plot(kind='bar')
            plt.title(f"{num_var} by {cat_var}")
            plt.ylabel("Average Value")


            messagebox.showinfo("Insight",
                f"Bar chart shows average '{num_var}' across categories of '{cat_var}'.")
        else:
            messagebox.showinfo("Info", "Categorical data has too many unique values.")


    else:
        messagebox.showinfo("Info", "No suitable visualization found.")


    plt.tight_layout()
    plt.show()



# ---------------- UI ----------------
title = tk.Label(root, text="Basic Visualization Assistant", font=("Arial", 16))
title.pack(pady=10)


upload_btn = tk.Button(root, text="Upload CSV / Excel File", command=upload_file)
upload_btn.pack(pady=8)


info_text = tk.Text(root, height=16, width=85)
info_text.pack(padx=10, pady=10)


analyze_btn = tk.Button(root, text="Analyze & Visualize", command=analyze_and_visualize)
analyze_btn.pack(pady=10)


root.mainloop()
```

➢ Conclusion –

- This project presents a simple and effective approach to automated data visualization using a rule-based system. The developed visualization assistant is capable of analyzing dataset structure, identifying data types, and selecting an appropriate graph without requiring manual intervention. By using basic statistical measures such as variance for numeric data and unique value count for categorical data, the system ensures that the chosen visualizations are both meaningful and easy to interpret.

- A key strength of this project lies in its simplicity and explainability. Unlike complex machine learning-based systems, the decision-making process here is transparent and easy to understand, making it especially suitable for beginners in data science. The integration of a graphical user interface further enhances usability by allowing users to interact with the system without needing programming knowledge.

- Although the system is limited to basic visualization types and does not incorporate domain-specific understanding, it successfully demonstrates how intelligent behavior can be achieved using simple rules and logical reasoning. This project provides a strong foundation for future improvements such as advanced visualization techniques, time-series analysis, and machine learning-based recommendation systems.

- In conclusion, the project achieves its objective of assisting beginners in selecting appropriate visualizations while maintaining a balance between functionality, simplicity, and interpretability.

THANK YOU !!!